Author: Pankaj Yadav
Date: 03/31/2026
Description:
This assignment is the refresher for python, R and statistical concepts learnt till now.

Week 1 Assignment : This assignment is a refresher on data analysis and visualization using Python and/or R. Find a data set that interests you and has appropriate data to create some interesting visualizations. A few good sources for finding datasets include Kaggle, UCI ML Repository, and the US Bureau of Labor Statistics.

With the dataset that you choose, perform the following steps using Python and/or R:

1. Write a summary of your data and identify at least two questions to explore visually with your data.
2. Create a histogram or bar graph from your data.
3. Create a boxplot from your data.
4. Create a bivariate plot from your data.
5. Create any additional visualizations that will help to answer the question(s) you want to answer.
6. Summarize your results and make a conclusion. Explain how you arrived at this conclusion and how your visualizations support your conclusion.

IMPORTANT: Please make sure to follow the following submission instructions carefully or it will result in having to resubmit or losing points if repeated incorrect submissions are made.

1. Write a summary of your data and identify at least two questions to explore visually with your data.

The dataset I chose is a loan application dataset which contains 614 rows and 13 columns. Every row in the dataset have a unique loan identifier and contains variable which provide information about applicants demographics (gender, marital status etc), their financial information and respective loan details. This dataset also contains the loan status variable which provides us the information if the loan was approved or not approved.

Questions to explore:

1. Lets check if there was a gender bias (male vs female with same income and credit history) while approving the loan?
2. How the applicants income and loan amounts vary when an application was approved vs rejected?

First thing first, lets import all the libraries that we would need and keep adding as we will write our code.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

Now lets use the read_csv function from pandas library to import the dataset from local folder `data` and create a data frame `loan_df`. As we do not need all the columns, I want to keep only the required

columns for our analysis, hence using `usecols` to restrict reading unwanted columns.

In [26]:
```
# import data from CSV file and create a DataFrame called loan_df and import only column
loan_df = pd.read_csv('data//Loan.csv'
                      , usecols=['Loan_ID', 'Gender', 'Married', 'ApplicantIncome', 'Coa
                                 , 'LoanAmount', 'Loan_Amount_Term', 'Loan_Status'])
```

Also lets make sure that the data is loaded in to data frame, so lets print first 3 rows

In [27]:
```
# display the first few rows of the DataFrame
print(loan_df.head(3))
```

```
    Loan_ID Gender Married  ApplicantIncome  CoapplicantIncome  LoanAmount  \
0  LP001002   Male      No             5849                0.0         NaN
1  LP001003   Male     Yes             4583             1508.0       128.0
2  LP001005   Male     Yes             3000                0.0        66.0

   Loan_Amount_Term Loan_Status
0             360.0           Y
1             360.0           N
2             360.0           Y
```

Next step, lets check if the columns and their data types are correctly loaded.

In [28]:
```
# display the column names and data types
print(loan_df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            614 non-null    object
 1   Gender             601 non-null    object
 2   Married            611 non-null    object
 3   ApplicantIncome    614 non-null    int64
 4   CoapplicantIncome  614 non-null    float64
 5   LoanAmount         592 non-null    float64
 6   Loan_Amount_Term   600 non-null    float64
 7   Loan_Status        614 non-null    object
dtypes: float64(3), int64(1), object(4)
memory usage: 38.5+ KB
None
```

As you see above columns like `Gender`, `Married`, `loanamount` etc have missing values in it. Also,there are rows with missing loan amount even though the loan is approved which does not make sense, hence I will drop these rows because I don't want to impute data for this column and most probably these are abandoned applications.

In [29]:
```
# taking care of missing values
print(loan_df.isnull().sum())
```

```
Loan_ID               0
Gender               13
Married               3
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount           22
Loan_Amount_Term     14
Loan_Status           0
dtype: int64
```

Lets remove the rows with missing `gender` and `Loan Amount`. Also, lets impute the 2 missing `Married` values with Mode loan amount term with the median values.

So what I am doing here. As I dont want to keep any missing values from `Gender` and `LoanAmount`, I am straight forward removing then in first code.

In the second line, I am selecting the `loan_amount_term` variable, and using the `fillna` function to fill in the missing values. I used the `mean()` function to get the mean of the column and impute it for missing.

Did similar with `Married` variable, but as its a classification variable, I used `Mode`.

In [32]:
```
# remove rows with missing values in gender and loan amount columns.
loan_df = loan_df.dropna(subset=['Gender', 'LoanAmount'])

#impute the mean of the loan amount term column and fill missing values with the mean
loan_df['Loan_Amount_Term'] = loan_df['Loan_Amount_Term'].fillna(loan_df['Loan_Amount_Te

# impute Married with the mode of the column and fill missing values with the mode
loan_df['Married'] = loan_df['Married'].fillna(loan_df['Married'].mode()[0])


print(loan_df.isnull().sum())
```

```
Loan_ID              0
Gender               0
Married              0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Loan_Status          0
dtype: int64
```

As we have resolved the `NaN` issue, lets describe our columns. NaN's can actually effect our final results , hence its always a good idea to resolve them first and then describe your data. The code below will create a descriptive statistical summary on each numeric variable within data frame.

In [33]:
```
# display summary statistics for the DataFrame
print(loan_df.describe())
```

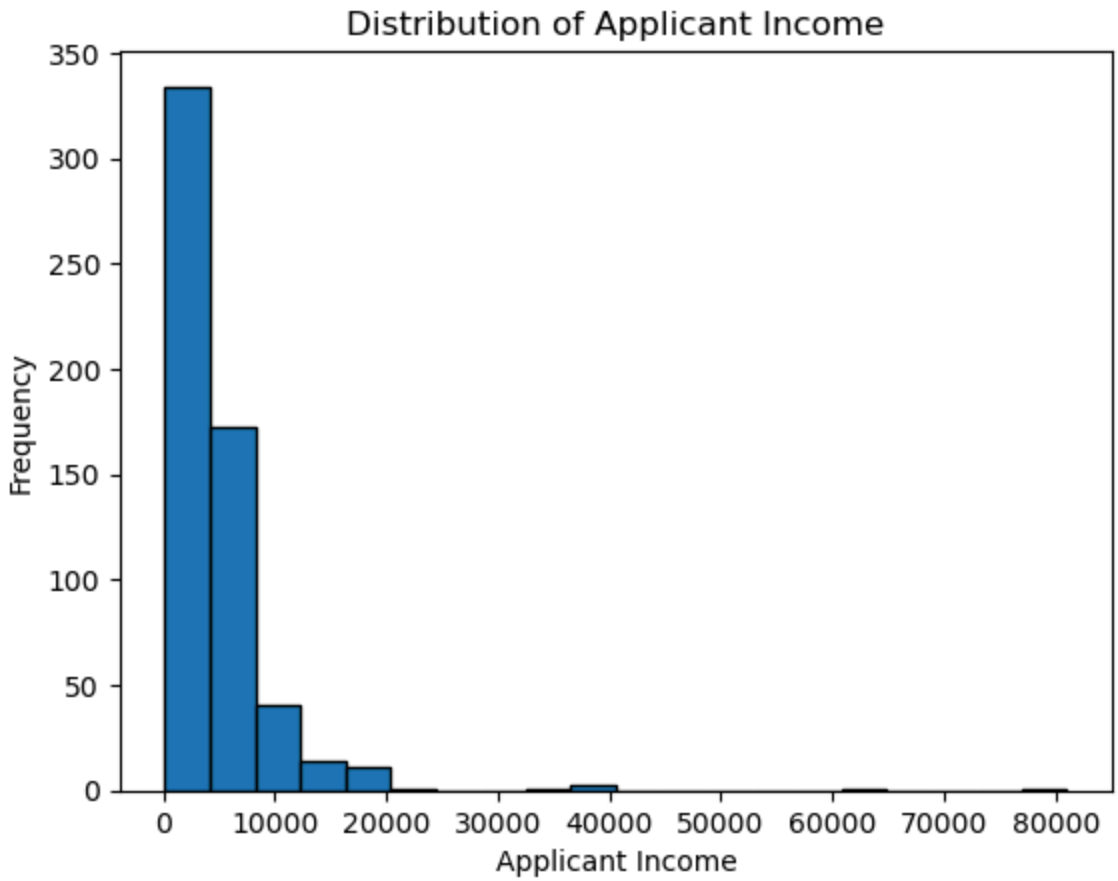
```
       ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term
count       579.000000         579.000000  579.000000        579.000000
mean       5293.630397        1646.606079  145.017271        342.159292
std        5843.467461        2990.502517   82.565103         64.966396
min         150.000000           0.000000    9.000000         12.000000
25%        2891.500000           0.000000  100.000000        360.000000
50%        3812.000000        1213.000000  127.000000        360.000000
75%        5736.000000        2331.500000  166.500000        360.000000
max       81000.000000       41667.000000  650.000000        480.000000
```

After dealing with missing values, the total count of rows is dropped from 614 to 579 records. If you look at the stats above, for `ApplicantIncome` and `CoapplicantIncome` the mean is much higher than median (50th percentile) which means, the data is right skewed there. Also, for `CoapplicantIncome`, the lower quartile shows most values as zero, indicating most co applicant don't have income. `LoanAmount` is also slightly right skewed when you look at the mean and 50th percentile values.

2. Create a histogram or bar graph from your data.

Lets create a histogram with `ApplicantIncome` to see the spread. As we noted above the data should show that its right skewed and should have a number of outliers in right. Lets code it and see the output. I am using `pyplot` from `matplotlib` library and its `hist` function to create a histogram. Set the bins to 20 to divide the data range in to 20 equal width and chose an edge colour as black to distinguish between two bars.

In [34]:
```
# create a histogram of the applicant income column
plt.hist(loan_df['ApplicantIncome'], bins=20, edgecolor='black')
plt.title('Distribution of Applicant Income')
plt.xlabel('Applicant Income')
plt.ylabel('Frequency')
plt.show()
```



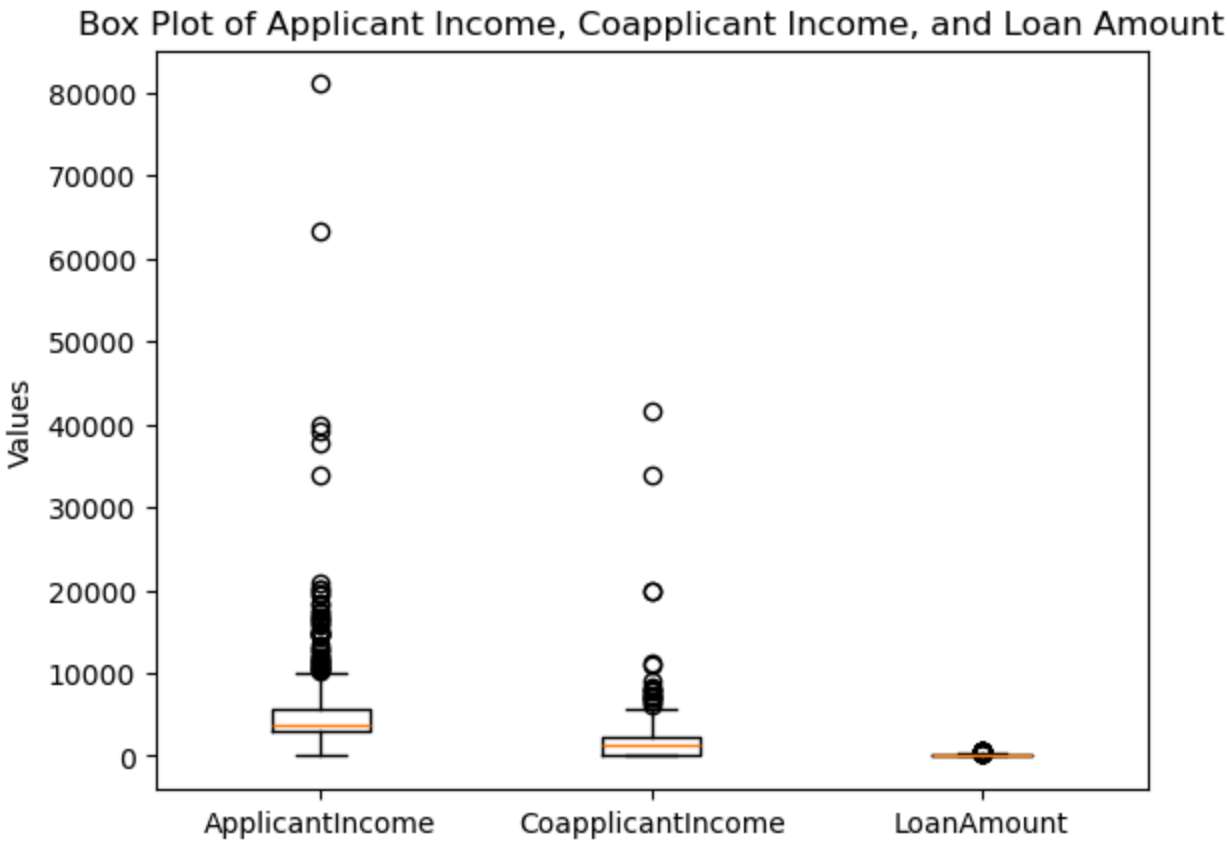
As expected, the distribution is highly skewed towards right with most of the applicants earning between 0 to ~5K with a few high earners earning up to 80K. These high earners are the outliers and there will be more which we can confirm using a box plot in next step.

3. Create a boxplot from your data.

So code I used to `boxplot` function from `pyplot` instead of `pandas` own `boxplot` because panda itself uses `matplotlib` at the backend to create the box plots.

So to understand the spread and outliers, boxplots are the best option for data visualizations. I have provided three variables `ApplicantIncome` , `CoapplicantIncome` and `LoanAmount` . The `tick_labels` just label visualization of each variable on `x axis` . Also, `plt.title` sets the title and `ylabel` labels the `Y-Axis` .

In [38]:
```
# Create a box plot of numeric columns to identify outliers
plt.boxplot([loan_df['ApplicantIncome'], loan_df['CoapplicantIncome'], loan_df['LoanAmou
            tick_labels=['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount'])
plt.title('Box Plot of Applicant Income, Coapplicant Income, and Loan Amount')
plt.ylabel('Values')
plt.show()
```



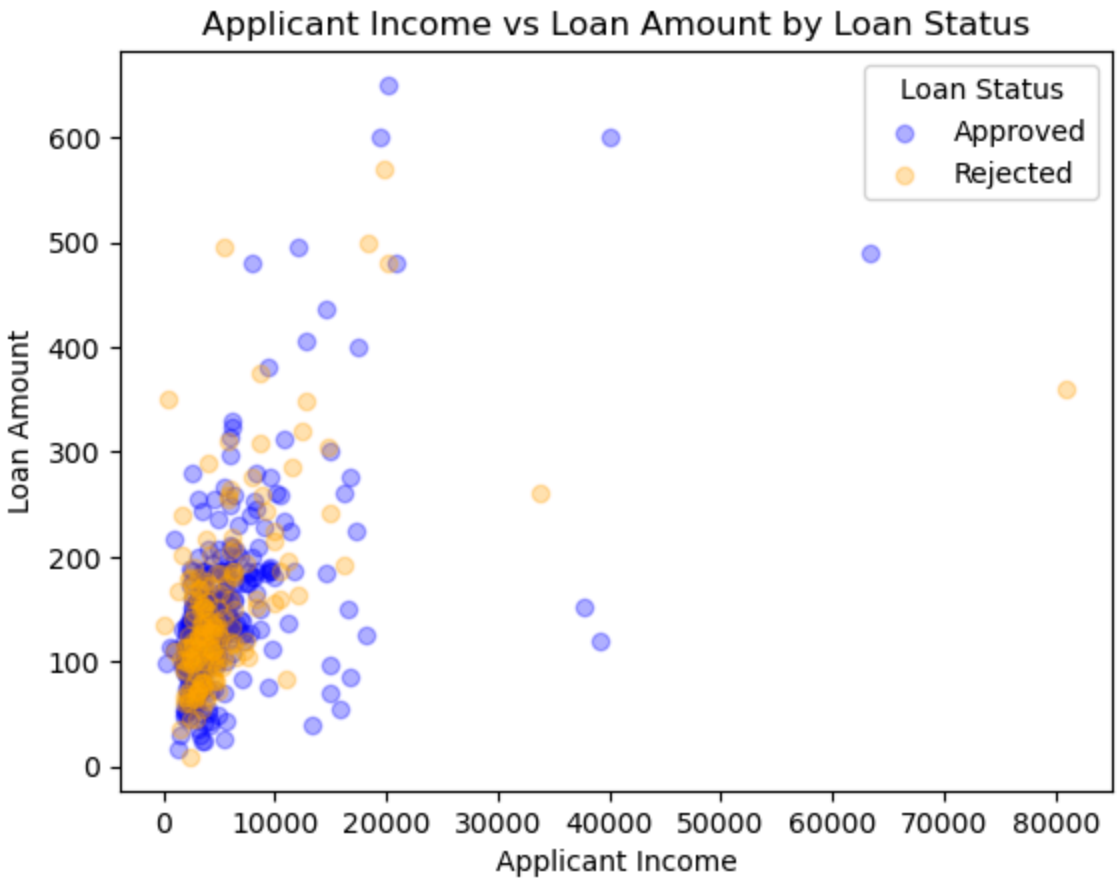
As expected from the statistical output above from describing the data frame, we see a lot of outliers in `applicantIncome` and `CoapplicantIncome` as you see in the form of dots extending above the upper ends.

4. Create a bivariate plot from your data.

How the applicants income and loan amounts vary when an application was approved vs rejected?

I wanted to check that how the `ApplicantIncome` relates to `LoanAmount` when a loan was approved or rejected. Creating a scatter plot using `pyplot.scatter` from `matplotlib`. So I created two data frames, one with approvals and other with rejections. Plotted the `loanAmount` and `ApplicantIncome` for each of these data frames. I chose `ApplicantIncome` on X axis and `LoanAmount` on Y Axis. A value of `alpha = 0.3` will add transparency so that we can see overlaps from other records too.

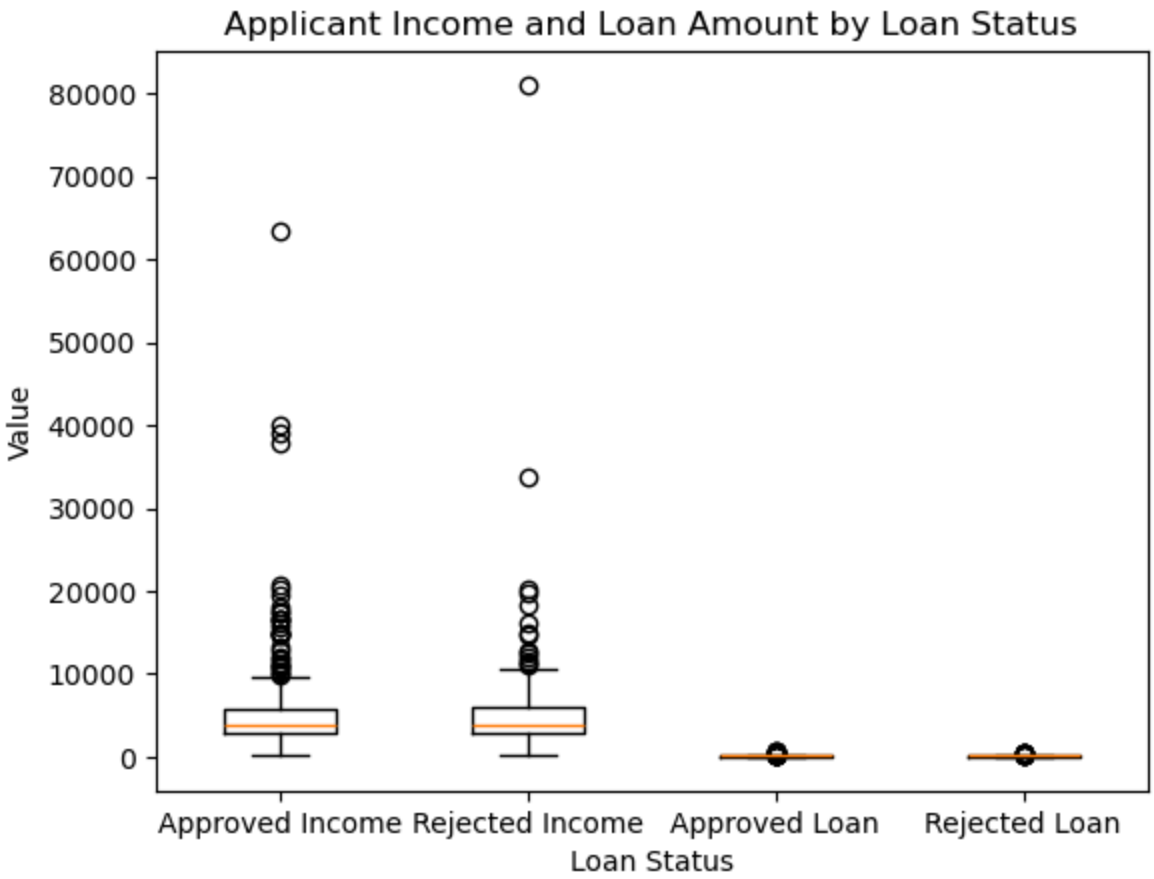
In [55]:
```
# create a scatter plot of applicant income and loan amount by loan status
approved = loan_df[loan_df['Loan_Status'] == 'Y']
rejected = loan_df[loan_df['Loan_Status'] == 'N']

plt.scatter(approved['ApplicantIncome'], approved['LoanAmount'], alpha=0.3, label='Appro
plt.scatter(rejected['ApplicantIncome'], rejected['LoanAmount'], alpha=0.3, label='Rejec
plt.title('Applicant Income vs Loan Amount by Loan Status')
plt.xlabel('Applicant Income')
plt.ylabel('Loan Amount')
plt.legend(title='Loan Status')
plt.show()
```



So in above Bi-Variate (two variable) scatter plot, each dot represents the value of that observation. The dots in `Orange` represents rejected loans and the `blue` approved loans. Looking at the plot it shows that most points in both approved and rejected loans are clustered at lower income and around lower to mid loan amounts without any clear separation between two. Also, most of the high income outliers show approved applications while the one with 80K salary shows a rejection. I really do not see any strong correlation between two in both approved and rejected applications. Lets see if a grouped box plot could help us understand the relationship better.

So the box plot is created with approved and rejected `ApplicantIncome` and `LoanAmount`. Used Matplotlib's pyplot.boxplot.

In [60]:
```
# grouped box plot between Applicant Income and Loan Amount by Loan Status
plt.boxplot([loan_df[loan_df['Loan_Status'] == 'Y']['ApplicantIncome'],
             loan_df[loan_df['Loan_Status'] == 'N']['ApplicantIncome'],
             loan_df[loan_df['Loan_Status'] == 'Y']['LoanAmount'],
             loan_df[loan_df['Loan_Status'] == 'N']['LoanAmount']],
            tick_labels=['Approved Income', 'Rejected Income', 'Approved Loan', 'Rejecte
plt.title('Applicant Income and Loan Amount by Loan Status')
plt.xlabel('Loan Status')
plt.ylabel('Value')
plt.show()
```



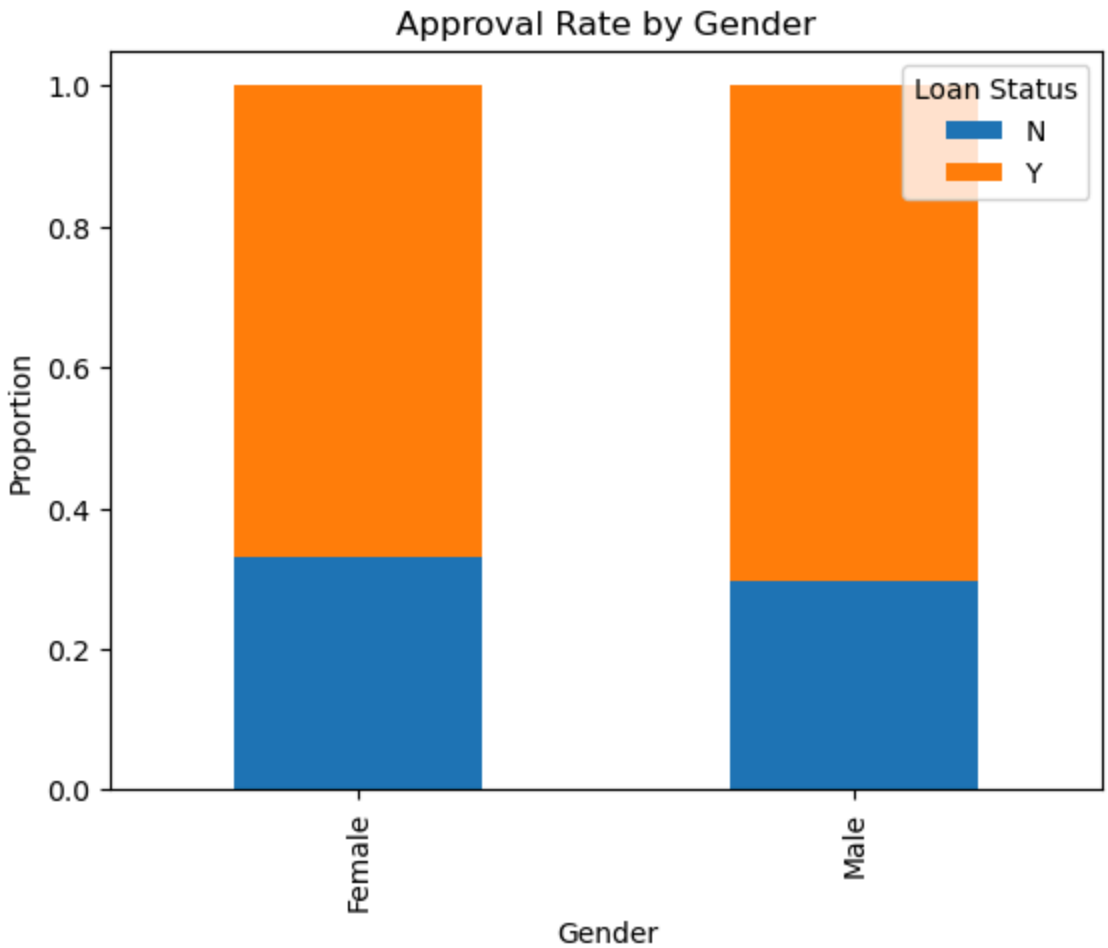
In the above plot you can see there are four subplots side by side. the first two plots on the left are applicant income for both approved and rejected applications, and the right two are for approved and rejected loan amounts. If you look at the approved and rejected income plots, they are very similar and have median at around same values. the approved loans have slightly higher outlier counts compared to rejected ones but rejected loa have the highest value of income for a rejected loan.So based on these two subplots, the income alone is not the main driver of approval and rejection of a loan. Similar results exists for loan amounts, the subplots are pretty packed and similar. This suggests that the income and loan amounts are not the only drivers for a loans approval.

5. Create any additional visualizations that will help to answer the question(s) you want to answer.

So the next chart that I would create is a stacked bar graph to check if there is a gender bias in approvals between two. To achieve this we will have to get the counts of approvals and rejections grouped by `genders` and respective `loans_status`. A `normalize = True` will help us normalize the data between teo groups by providing proportions instead of just counts. The `unstack` will create a formatted table with proportions out of grouped data. I used the pandas plot function due to its defaults for axis labelling and range.

In [61]:
```
# bar chart with approval rate by gender
approval_rate_by_gender = loan_df.groupby('Gender')['Loan_Status'].value_counts(normaliz
approval_rate_by_gender.plot(kind='bar', stacked=True)
plt.title('Approval Rate by Gender')
plt.xlabel('Gender')
plt.ylabel('Proportion')
```

```
plt.legend(title='Loan Status', loc='upper right')
plt.show()
```



Looking at the chart, the approval and rejection rates are fairly similar with very slight differences. If the approval proportion for one gender is very high compared to other then it could be considered a biased loan approval system, but I do not see any such difference between two. I would also like to mention that this is very preliminary analysis which could not affirm the bias and there could be multiple other factors that could prove or disprove the bias between two.

6. Summarize your results and make a conclusion. Explain how you arrived at this conclusion and how your visualizations support your conclusion.

I chose the loan application data from Kaggle.com and posed two questions at the start of the assignment. While doing the analysis, I cleaned up the data, dropped or imputed values as required and finally had 579 rows. The descriptive statistics on variable showed that both income variables and loan amount are right skewed with outliers.

The histogram and box plots showed that the most applicant falls under lower income range with a few outliers have higher incomes. The bivariate scatter plot and the box plot showed that there is a substantial overlap between rejected and approved applications for both loan amount and applicant incomes. the spread and median values in box plot were similar hence suggesting that the income and loan amounts are not influencing alone the final decisioning of the application.

The stacked bar chart which was to answer my gender bias question between male and female applicants approval rates also did not show significant evidence of gender bias between two.

Hence based on preliminary analysis and visualizations, my overall conclusion is that the loan approval or rejection are not driven primarily by applicants income and loan amounts and there is no major gender bias visible between the approved and rejected loans as well.